

TRABAJO DE INVESTIGACIÓN: ENTORNOS DE DESPLIEGUE Y ARQUITECTURA DE APLICACIONES

Módulo: Desarrollo de Aplicaciones Multiplataforma

Curso: 2025-2026

Indice

- 1. Introducción
- 3. Contenidos que se deben investigar
 - 3.1 Entorno de ejecución de aplicaciones Java
 - 3.2 Creación de instaladores
 - 3.3 Servicios, APIs y comunicación en red
 - 3.4 Contenedores con Docker
 - 3.5 Aplicaciones frontend con Angular y React

1. Introducción

El ciclo de vida de una aplicación no termina al escribir su código; la fase de despliegue es crítica para asegurar que funcione correctamente en servidores o equipos reales. Este proceso exige coordinar el sistema operativo, la configuración de red y la infraestructura de hardware.

El objetivo de este trabajo es investigar y documentar, desde el enfoque de Sistemas Informáticos, los elementos clave en la instalación, ejecución y comunicación de software actual. Para ello, se analizará el backend en Java (Spring Boot) y el frontend en Angular y React.

A lo largo del documento se examinarán los requisitos de la máquina virtual (JVM), la creación de instaladores, la comunicación mediante APIs (HTTP/HTTPS), el aislamiento con contenedores Docker y el despliegue de interfaces web. El resultado ofrece una visión técnica, clara y estructurada de los entornos de explotación reales.

3.1 Entorno de ejecución de aplicaciones Java

- Investiga qué necesita un equipo para ejecutar aplicaciones Java. Explica qué es la JVM y por qué permite ejecutar programas Java en distintos sistemas operativos.

Para que un ordenador pueda ejecutar aplicaciones Java, es necesario que tenga instalado el entorno adecuado. Java fue diseñado para que los programas puedan funcionar en distintos sistemas operativos sin necesidad de modificar el código, ya sea en Windows, Linux o macOS. Esto se consigue gracias a la JVM (Java Virtual Machine o Máquina Virtual de Java), que es una especie de intermediario entre el programa y el sistema operativo. Cuando se desarrolla una aplicación Java, el código se compila en bytecode, y la JVM se encarga de interpretarlo para que pueda ejecutarse correctamente en cada plataforma.

- Diferencia entre JDK y JRE... Indica cuál se necesita para desarrollar y cuál para ejecutar aplicaciones.

Dentro de Java existen dos componentes importantes que suelen confundirse: el JDK y el JRE. El JDK (Java Development Kit) es el paquete completo que necesitan los desarrolladores, ya que incluye herramientas como el compilador, bibliotecas y utilidades para crear programas. Por otro lado, el JRE (Java Runtime Environment) contiene únicamente lo necesario para ejecutar aplicaciones Java ya creadas. En resumen, para programar se necesita el JDK, mientras que para usar aplicaciones Java basta con el JRE.

- Describe los pasos básicos para instalar Java en Windows o Linux y cómo comprobar la instalación desde la terminal con comandos como `java -version`.

La instalación de Java es relativamente sencilla. En Windows, normalmente se descarga el instalador desde la página oficial de Oracle o desde una distribución de OpenJDK, se ejecuta y se siguen los pasos del asistente. En Linux, suele instalarse mediante comandos desde la terminal utilizando gestores de paquetes. Una vez instalado, es recomendable comprobar que funciona correctamente con comandos como `java -version` o `javac -version`.

- Explica variables JAVA_HOME y PATH... para qué sirven y qué problemas pueden aparecer si no están configuradas correctamente.

Además de la instalación, hay variables de entorno que son muy importantes, especialmente JAVA_HOME y PATH. JAVA_HOME indica la ubicación donde está instalado Java en el sistema, mientras que PATH permite que comandos como java o javac puedan ejecutarse desde cualquier carpeta de la terminal. Si estas variables no están bien configuradas, pueden aparecer problemas como que el sistema no reconozca Java, errores al compilar programas o conflictos entre distintas versiones instaladas.

3.2 Creación de instaladores

- Define qué es un instalador y qué ventajas tiene frente a entregar simplemente un archivo comprimido o un .jar.

Un instalador es un archivo ejecutable (.exe en Windows, .dmg o .pkg en macOS, o un script de instalación en Linux) diseñado para automatizar el proceso de colocar los archivos de un programa en los directorios correctos, configurar el sistema operativo y preparar la aplicación para ser utilizada por el usuario final. A diferencia de un archivo comprimido (.zip, .rar) o un archivo Java (.jar), que suelen requerir una instalación portable o manual, el instalador guía al usuario a través de un asistente (wizard) para realizar tareas complejas en segundo plano.

- Investiga herramientas que permitan generar instaladores o ejecutables para aplicaciones Java, por ejemplo Inno Setup, install4j, jpackage o Launch4j.

Para distribuir una aplicación Java de forma profesional en 2026, la elección de la herramienta depende de si buscas algo integrado en el JDK, una solución comercial multiplataforma o un wrapper sencillo para Windows.

1. jpackage (La opción oficial): Introducido de forma permanente en el JDK 16 y ahora consolidado en el JDK 26, es la herramienta estándar de Java para crear paquetes "autocontenidos".
2. install4j (La opción profesional/comercial): Considerada por muchos como la herramienta más potente del mercado para aplicaciones empresariales.
3. Launch4j (El "Wrapper" clásico): Si solo necesitas que tu archivo .jar se comporte como un .exe de Windows, esta sigue siendo una opción ligera y popular.
4. Inno Setup (El estándar de Windows): Aunque no es una herramienta específica para Java, es el motor de instalación preferido para Windows debido a su flexibilidad y gratuidad.

- Explica qué elementos puede incluir un instalador: archivos de la aplicación, accesos directos, configuración inicial, dependencias, desinstalador y rutas de instalación.

Un instalador es mucho más que un simple "copiador de archivos". Su función principal es preparar el ecosistema del sistema operativo para que la aplicación funcione correctamente desde el primer segundo.

1. Archivos de la aplicación (Payload): Es el núcleo del instalador. Contiene todo el software ejecutable y sus recursos necesarios.
2. Rutas de instalación: El instalador debe decidir (o preguntar) dónde residirá la aplicación.
3. Dependencias y Entorno de Ejecución (Runtime): Este es un punto crítico para aplicaciones Java.
4. Accesos directos y Menús: Para facilitar la experiencia del usuario (UX), el instalador interactúa con la interfaz del sistema operativo.
5. Configuración inicial y Registro: Antes de que el usuario abra la app, el instalador realiza tareas de "fontanería".
6. Desinstalador (Uninstaller): Un buen instalador siempre debe incluir su contraparte.

3.3 Servicios, APIs y comunicación en red

- **Explica qué es una API y qué papel tiene dentro del modelo cliente-servidor. El objetivo es entender la comunicación entre aplicaciones, no programar una API completa.**

(Interfaz de Programación de Aplicaciones) es un conjunto de reglas, protocolos y herramientas que permite que dos aplicaciones de software se comuniquen e intercambien datos entre sí. Funciona como un intermediario o "puente" que facilita la integración de sistemas y funcionalidades sin necesidad de conocer su funcionamiento interno. En cuanto al modelo cliente servidor la API tiene el papel de determinar qué funcionalidades puede acceder el cliente (validar permisos) procesa los datos pedidos por el cliente y devuelve la información actuando como intermediario.

- **Investiga qué es Spring Boot y por qué se utiliza para crear servicios o APIs en Java.** Spring Boot es un framework basado en Spring Framework que sirve para desarrollar aplicaciones en Java de forma más rápida y sencilla, especialmente servicios web y APIs REST. Además de simplificar el desarrollo, Spring Boot automatiza mucha configuración que antes había que hacer manualmente.

- **Describe conceptos básicos de red relacionados con una API: dirección IP, puerto, protocolo HTTP/HTTPS, peticiones GET y POST, y respuesta del servidor.**

Dirección ip: Es un identificador único que tiene cada dispositivo conectado a una red. Sirve para localizar y comunicar equipos entre sí en Internet o en una red local. Una API necesita una dirección IP para que los clientes puedan encontrar el servidor donde está alojada.

Puerto: Es un número que identifica un servicio o aplicación dentro de un dispositivo. Permite que varios servicios funcionen al mismo tiempo en la misma dirección IP. Por ejemplo, una API web suele utilizar los puertos 80 (HTTP) o 443 (HTTPS).

Protocolo HTTP/HTTPS: Es el conjunto de reglas que permite la comunicación entre cliente y servidor en la web.

-HTTP (HyperText Transfer Protocol): transmite información sin cifrar.

-HTTPS: es la versión segura de HTTP, ya que cifra los datos mediante SSL/TLS para proteger la comunicación.

Las APIs modernas normalmente utilizan HTTPS para garantizar seguridad.

peticiones GET y POST:

GET se utiliza para solicitar o consultar información al servidor sin modificar datos.

Ejemplo: obtener una lista de usuarios.

POST: se utiliza para enviar datos al servidor y crear o modificar información.

Ejemplo: registrar un nuevo usuario.

Respuesta del servidor:

Es la información que el servidor devuelve al cliente después de recibir una petición.

Incluye: Un código de estado HTTP (200, 404, 500, etc.) los datos solicitados o un mensaje de error.

3.4 Contenedores con Docker

Explica qué es Docker y qué problema intenta resolver: ejecutar aplicaciones en entornos aislados y reproducibles.

Docker es una plataforma que permite crear y ejecutar aplicaciones dentro de contenedores. El problema que intenta resolver es el de la compatibilidad entre entornos. Docker crea un entorno aislado con el que se lleva todo lo necesario para poder ejecutar aplicaciones en distintos dispositivos evitando fallos de ejecución de esta manera.

- **Diferencia entre máquina virtual y contenedor.**

Una máquina virtual simula un ordenador completo. Mientras que un contenedor comparte el sistema operativo del host, pero mantiene aislada la aplicación y sus dependencias.

Indica ventajas e inconvenientes de cada enfoque de forma sencilla.

Como ventajas el contenedor es más ligero, más fácil de pasar entre equipos, consume menos memoria y es ideal para microservicios y despliegues rápidos.

Como inconvenientes está menos aislado que una máquina virtual y dependen del servidor del host para funcionar.

- **Define los conceptos básicos: imagen, contenedor, Dockerfile, volumen y puerto expuesto.**

-Imagen: Una imagen es una plantilla que contiene todo lo necesario para ejecutar una aplicación. La imagen no puede cambiar y sirve como base para crear contenedores.

-Contenedor: Un contenedor es una instancia en ejecución de una imagen. Es como la aplicación funcionando y dentro esta la plantilla que es como la imagen.

-Dockerfile: Un Dockerfile es un archivo de texto con instrucciones para crear una imagen Docker.

-Volumen: Un volumen permite guardar datos fuera del contenedor.

Esto es importante porque los contenedores pueden eliminarse y recrearse.

Gracias a los volúmenes, los datos permanecen guardados aunque el contenedor desaparezca.

-Puerto expuesto: Un contenedor puede tener servicios funcionando internamente, pero para acceder desde fuera se necesita exponer un puerto. Así el navegador puede acceder a la aplicación.

• **Relaciona Docker con Java: cómo podría empaquetarse una aplicación Spring Boot en una imagen y ejecutarse en cualquier equipo con Docker instalado.**

Puede empaquetarse dentro de una imagen Docker para ejecutarse en cualquier ordenador que tenga Docker instalado, sin importar el sistema operativo y demás.

Los pasos serían:

- Se desarrolla la aplicación Spring Boot.
- Se genera el archivo .jar.
- Se crea un Dockerfile.
- Docker construye una imagen.
- La imagen puede ejecutarse en cualquier equipo con Docker.

3.5 Aplicaciones frontend con Angular y React

• **Explica qué es una aplicación frontend y su papel en cliente-servidor.**

Una aplicación frontend es la parte visual de una aplicación o página web, es decir, la interfaz con la que interactúa el usuario. Su papel en cliente-servidor es que como cliente se muestra la interfaz, se piden cosas y se espera una respuesta del servidor que es el que accede a base de datos, valida información enviada desde el front y envía las respuestas correspondientes a las llamadas mediante una API.

• **Investiga Angular y React.**

Angular es un framework frontend desarrollado por Google. Se utiliza para crear aplicaciones web modernas y complejas. Está basado en TypeScript y cuenta con una estructura sólida y organizada lo que la hace genial para proyectos de empresa aunque es

un framework algo pesado eso se contrasta con su gran variedad de funcionalidades que incluye de serie.

React es una librería JavaScript creada por Meta. Se centra en construir interfaces mediante componentes reutilizables. Es una librería muy popular que cuenta con una comunidad enorme y por tanto una gran presencia en el sector debido sobre todo a su rendimiento gracias al DOM virtual aparte en caso de necesitar algo a la hora del desarrollo puedes ayudarte de herramientas para usar funciones y es muy flexible por tanto puedes variar la estructura a tu antojo sin que realmente haya un problema.

- **Describe ejecución: Node.js, npm, servidor dev.**

- Node.js es un entorno que permite ejecutar JavaScript fuera del navegador. Por ejemplo Angular y React necesitan Node.js para funcionar durante el desarrollo.

- npm es el gestor de paquetes de Node.js. Sirve para: instalar librerías, descargar dependencias, actualizar paquetes y ejecutar scripts.

- (como curiosidad actualmente npm install es peligroso de utilizar actualmente por que recientemente se vio que inyectaban código perjudicial para el usuario que instala archivos no deseables en tu equipo. Como alternativa se recomienda usar pnpm que es una versión que no te instala las versiones que no tengan más de 1 día de antigüedad y aparte tiene un sistema de seguridad el cual evita la instalación de archivos. La fuente de esta noticia es el Youtuber Midudev)

- Un servidor de desarrollo es una herramienta que ejecuta la aplicación localmente mientras se programa. Este permite ver cambios en tiempo real, recargar automáticamente la página y probar la aplicación antes de publicarla.

- **Explica build y archivos generados.**

- El build es el proceso mediante el cual una aplicación frontend se prepara para ser publicada o desplegada. Durante este proceso el código se compila, se optimiza y se reducen tamaños de archivos. El objetivo de este proceso es que la aplicación cargue más rápido.

- **Describe despliegue en servidor web.**

- El despliegue consiste en publicar la aplicación en un servidor para que los usuarios puedan acceder desde internet.

- **Relación con backend Java y APIs.**

- El backend Spring Boot: recibe la petición, valida datos, consulta la base de datos y devuelve una respuesta JSON.

Es como si la API (intermediario) usase la lógica del backend java accediendo a ella mediante endpoints y así realizar todas las funciones mencionadas.